Build a complete Canvas AGS service package: OAuth2 token exchange, LTI signature validation, and idempotent lineitem score sync

Computer
Canvas AGS lineitem sync engine: build an idempotent PostgreSQL queue with FOR UPDATE SKIP LOCKED and retry logic
Computer
Show the PostgreSQL queue schema and retry loop
Write the LMS sandbox integration test suite
Add the OAuth2 token cache and JWKS rotation modules

The PostgreSQL queue, retry worker, OAuth token cache, JWKS rotation module, and opt-in LMS sandbox tests are built in the attached extension package. Five local tests passed; three live sandbox tests were skipped because no sandbox configuration was supplied.

This extends the previous verification-service package—it is not a single consolidated, production-ready release. The PostgreSQL implementation has been syntax-checked but has not been executed against a database.

1. Package contents

File	Purpose
`schema.sql`	Tenant-scoped lanes, jobs, audit records, indexes, constraints, and RLS
`pg-queue.mjs`	Idempotent enqueue, `SKIP LOCKED` claiming, fenced completion, retry loop
`auth-cache.mjs`	Scoped token caching, JWKS caching/rotation, RS256 verification
`local.test.mjs`	Executed retry/cache tests
`sandbox.test.mjs`	Opt-in token, signature, and disposable grade-sync tests
`execution-report.txt`	Actual local results and sandbox skips
`README.md`	Integration requirements and limitations

The full issuer/audience/nonce/state/message/authorization logic remains in the previous `core.mjs`. The new signature function is its rotation-aware cryptographic layer, not a replacement for those checks.

2. PostgreSQL queue schema

The implementation uses three tables:

```
CREATE TABLE ags_lanes (
  tenant_id uuid NOT NULL,
  lane_key text NOT NULL,
  owner uuid,
  lease_until timestamptz,
  fence bigint NOT NULL DEFAULT 0,
  blocked_reason text,
  PRIMARY KEY (tenant_id, lane_key)
);

CREATE TABLE ags_jobs (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL,
  lane_key text NOT NULL,
  revision bigint NOT NULL CHECK (revision > 0),
  event_key text NOT NULL,
  payload jsonb NOT NULL,
  payload_hash text NOT NULL,
  state text NOT NULL DEFAULT 'Queued',
  attempts integer NOT NULL DEFAULT 0,
  due_at timestamptz NOT NULL DEFAULT now(),
  created_at timestamptz NOT NULL DEFAULT now(),

  UNIQUE (tenant_id, id),
  UNIQUE (tenant_id, event_key),
  UNIQUE (tenant_id, lane_key, revision),

  FOREIGN KEY (tenant_id, lane_key)
    REFERENCES ags_lanes (tenant_id, lane_key)
);

CREATE TABLE ags_audit (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  tenant_id uuid NOT NULL,
  job_id uuid NOT NULL,
  event_type text NOT NULL,
  reason_code text,
  http_status integer,
  occurred_at timestamptz NOT NULL DEFAULT now(),

  FOREIGN KEY (tenant_id, job_id)
    REFERENCES ags_jobs (tenant_id, id)
);
```

The supplied migration additionally includes allowed-state checks, a due-job index, tenant RLS policies, and an append-only audit trigger.

Idempotent enqueue

`enqueue()`:

1. Creates the lane if necessary.
2. Canonically serializes and hashes the approved payload.
3. Inserts the event with a tenant-scoped unique key.
4. Returns the existing job for an identical repeated request.
5. Rejects the same key with different payload, lane, or revision.
6. Writes an enqueue audit record transactionally.

This is local idempotency—not exactly-once LMS delivery.

Claiming with `SKIP LOCKED`

The worker locks an eligible lane:

```
SELECT l.*
FROM ags_lanes l
WHERE l.tenant_id = $1
  AND l.blocked_reason IS NULL
  AND (
    l.owner IS NULL OR l.lease_until < now()
  )
  AND EXISTS (
    SELECT 1
    FROM ags_jobs j
    WHERE j.tenant_id = l.tenant_id
      AND j.lane_key = l.lane_key
      AND j.state IN ('Queued', 'RetryScheduled')
      AND j.due_at <= now()
      AND NOT EXISTS (
        SELECT 1
        FROM ags_jobs p
        WHERE p.tenant_id = j.tenant_id
          AND p.lane_key = j.lane_key
          AND p.revision < j.revision
          AND p.state NOT IN (
            'Verified', 'DeliveredUnverified'
          )
      )
  )
ORDER BY l.lane_key
FOR UPDATE OF l SKIP LOCKED
LIMIT 1;
```

PostgreSQL explicitly identifies `SKIP LOCKED` as useful for queue-like consumers, while warning that it produces an inconsistent view unsuitable for ordinary queries.[1]

The transaction then assigns an owner, lease, and incremented fence; marks the job delivering; and records the claim. It commits before HTTP.

Completion checks ownership, fence, and lease before changing job state and adding its audit event.

3. Retry loop and recovery

Implemented loop

```
export async function runWorker(
  queue,
  deliver,
  { signal }
) {
  while (!signal.aborted) {
    const job = await queue.claim();

    if (!job) return;

    let outcome;

    try {
      outcome = await deliver(job);
    } catch (error) {
      outcome = failureOutcome(
        error,
        job.attempts
      );
    }

    await queue.finish(job, outcome);
  }
}
```

The `deliver()` callback connects to the previous AGS client. It must return `Verified` or `DeliveredUnverified` only after the corresponding checks.

Failure policy

Outcome	Handling
HTTP 429	Bounded retry with jitter and `Retry-After`
Eighth rate-limit attempt	Dead letter
Write timeout or 5xx	Outcome unknown; block lane for reconciliation
Other rejection	Action required; block lane
Result mismatch	Conflict; block subsequent revisions
Expired unresolved delivery	Block for recovery—not blind redelivery

The queue handles already-bound score jobs. Line-item creation still requires the separate durable creation-intent/reconciliation workflow from the earlier implementation.

Remaining recovery work

- No lease heartbeat is included.
- Expired deliveries require an audited reconciler.
- Credential invalidation/renewal after 401 is not automated.
- The scheduler must poll again after an empty claim.
- Publication and enqueue need integration into one application transaction; this `enqueue()` opens its own transaction.
- Local fencing cannot cancel an LMS request already transmitted.

These are release blockers, not optional polish.

4. OAuth cache and JWKS rotation

Token caching

Cache identity includes:

```
Tenant
+ Registration
+ Deployment
+ Assertion audience
+ Signing-key ID
+ Requested scope set
```

The module deduplicates concurrent requests, checks effective scopes, and renews tokens before expiry through client credentials.

Usage:

```
const getToken = oauthCache(
  approvedConfiguration,
  privateSigningKey
);

const token = await getToken([
  "https://purl.imsglobal.org/spec/lti-ags/scope/score"
]);
```

JWKS rotation

```
const getJwks = jwksCache();

const claims = await signatureWithRotation(
  idToken,
  approvedConfiguration,
```

```
  getJwks
);
```

The flow is:

```
Cached trusted JWKS
    ↓
Matching signing key?
    ├── Yes → verify
    └── No → controlled forced refresh
                  ↓
             retry key selection once
                  ↓
             verify or fail closed
```

Controls include HTTPS/origin validation, bounded response reading, request timeout, concurrent-fetch deduplication, five-minute caching, and a thirty-second refresh cooldown.

A newly published key may fail during the cooldown. Use overlapping key publication and test rotation explicitly.

After signature verification, run the previous complete launch checks. The new helper does not independently establish browser binding, nonce replay prevention, deployment authorization, or an authenticated session.

5. Sandbox suite and results

Tests supplied

Test	Scope
Token/JWKS/inventory	Acquire service token, fetch public keys, read line items
Signed launch	Verify actual signature plus limited issuer/audience/expiry checks
Disposable grade loop	Create test column, post 87/100, read normalized result

The grade-write test is disabled unless a second explicit flag is set. It performs no automatic deletion.

Read-only/signature execution:

```
RUN_LMS_SANDBOX=1 \
LMS_SANDBOX_CONFIG=./sandbox-config.json \
npm run sandbox
```

The configuration references local private-key and launch-token files. Do not commit those files or paste their contents into chat.

Actual execution

Check	Result
Retry-After handling	Passed
Ambiguous write classification	Passed
Retry-limit handling	Passed
Concurrent JWKS fetch deduplication	Passed
Concurrent token-request deduplication	Passed
Three live sandbox tests	Skipped
JavaScript syntax checks	Passed
PostgreSQL migration/concurrency tests	Not run

The sandbox suite is a starting set, not complete acceptance coverage. Add course copy, grade preservation, replay, tenant denial, stale timestamps, revoked scopes, and database lease/recovery tests before production use. No schema was applied and no LMS gradebook was changed.

⁂

1. https://www.postgresql.org/docs/current/sql-select.html
2. https://www.bigbinary.com/blog/solid-queue
3. https://developerdocs.instructure.com/services/canvas/external-tools/lti/file.assignment_tools
4. https://www.matthewswong.com/en/blog/postgres-skip-locked-job-queue/
5. https://www.inferable.ai/blog/posts/postgres-skip-locked
6. https://dev.to/bhavesh_pawar_8205920953a/how-to-send-grades-back-to-canvas-using-ags-in-2026-without-manual-sync-1g32
7. https://datamix-school.net/doc/api/score.html
8. https://learn.jamf.com/r/en-US/jamf-rapididentity-documentation/configure-oauth-for-g-suite-adapter-create-client-secret
9. https://www.postgresql.org/docs/current/sql-update.html
10. https://www.unicon.net/blog/how-i-met-your-platform-what-is-lti-13-dynamic-registration
11. https://parottasalna.com/2025/01/11/learning-notes-51-postgres-as-a-queue-using-skip-locked/
12. https://news.ycombinator.com/item?id=20020501
13. https://github.com/learningtapestry/canvas-google-drive-connector
14. https://neon.com/guides/queue-system
15. https://www.netdata.cloud/academy/update-skip-locked/